

SE3082 – Parallel Computing

Exercise 1 Details, Code & Output Verification



IT24103470
Kathisan A

Exercise 1 Details, Code & Output Verification

1. Exercise Task Description

Run the provided C code snippet using OpenMP and briefly describe how the `#pragma omp barrier` command works in this program.

2. Complete C Source Code (exercise1.c)

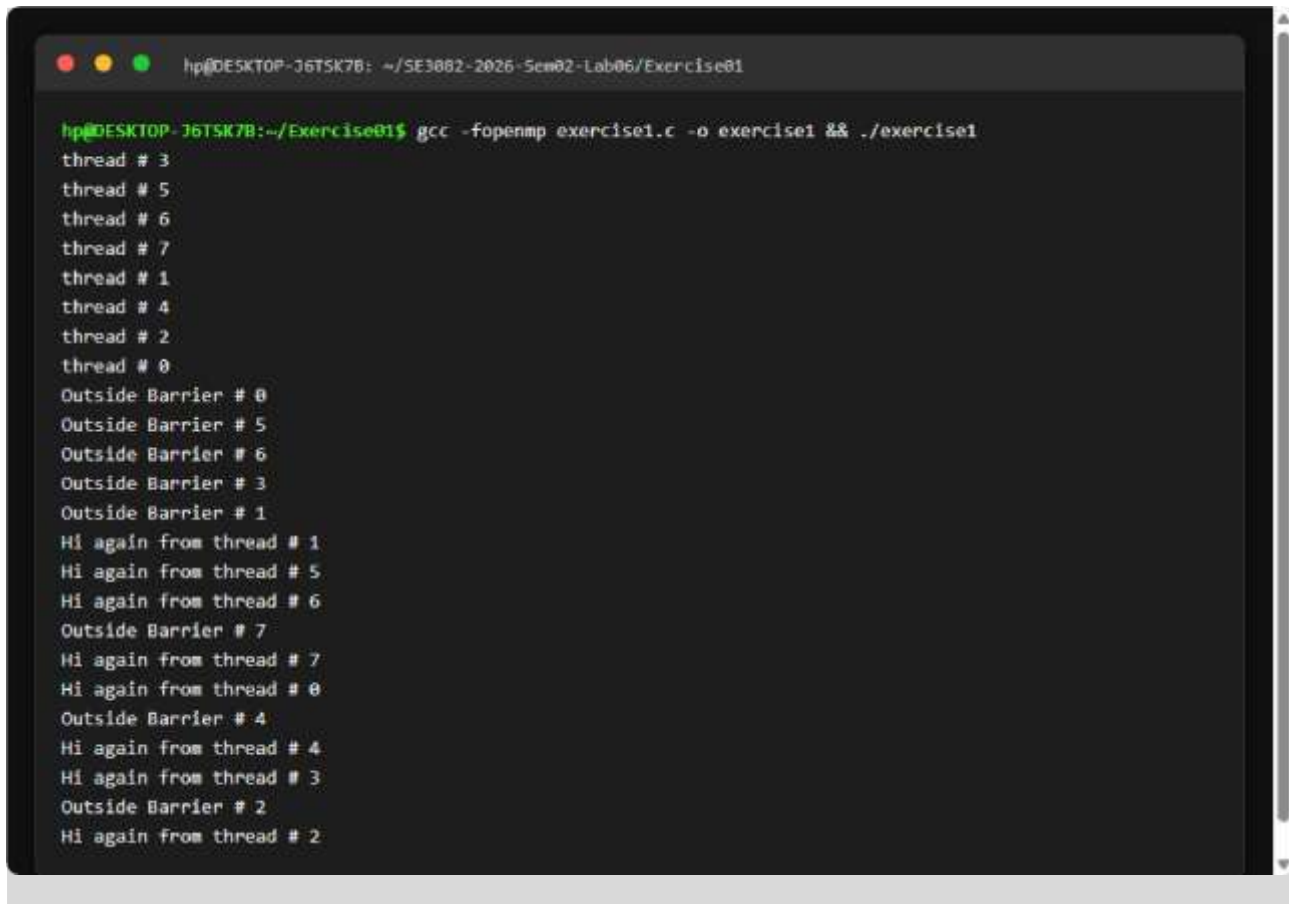
```
#include <omp.h>
#include <stdio.h>

int main() {
#pragma omp parallel
{
    int th = omp_get_thread_num();
    printf("thread # %d\n", th);

    #pragma omp barrier

    printf("Outside Barrier # %d\n", omp_get_thread_num());
    printf("Hi again from thread # %d\n", omp_get_thread_num());
}
    return 0;
}
```

3. Terminal Execution Output Screenshot

A terminal window with a dark background and light green text. The window title is 'hp@DESKTOP-36TSK7B: ~/SE3882-2026-Sem02-Lab06/Exercise01'. The command executed is 'gcc -fopenmp exercise1.c -o exercise1 && ./exercise1'. The output shows thread IDs (3, 5, 6, 7, 1, 4, 2, 0) printed in a non-sequential order, followed by 'Outside Barrier' messages for each thread, and then 'Hi again from thread' messages for each thread, also in a non-sequential order. This demonstrates that threads execute in parallel and are synchronized at the barrier.

```
hp@DESKTOP-36TSK7B: ~/SE3882-2026-Sem02-Lab06/Exercise01
hp@DESKTOP-36TSK7B:~/Exercise01$ gcc -fopenmp exercise1.c -o exercise1 && ./exercise1
thread # 3
thread # 5
thread # 6
thread # 7
thread # 1
thread # 4
thread # 2
thread # 0
Outside Barrier # 0
Outside Barrier # 5
Outside Barrier # 6
Outside Barrier # 3
Outside Barrier # 1
Hi again from thread # 1
Hi again from thread # 5
Hi again from thread # 6
Outside Barrier # 7
Hi again from thread # 7
Hi again from thread # 0
Outside Barrier # 4
Hi again from thread # 4
Hi again from thread # 3
Outside Barrier # 2
Hi again from thread # 2
```

4. Directive Analysis & Explanation

How #pragma omp barrier works:

The #pragma omp barrier directive acts as an explicit synchronization checkpoint. When threads enter the #pragma omp parallel region, each thread independently prints its thread ID. Upon encountering the barrier directive, every thread pauses and waits. No thread is permitted to cross the barrier until all threads in the parallel team have reached the barrier. Once the final thread arrives, all threads resume execution concurrently to print "Outside Barrier" and "Hi again from thread...".